Design and Analysis of Computer Algorithms

01:198:344

Spring 2026, Rutgers

Instructor: Surya Teja Gavva

Selection and QuickSort

- **Quick SELECT:** Uses a random pivot to partition.
  - Recurse on *one* side only.
  - **Complexity:** $\Theta(n)$ expected, $\Theta(n^2)$ worst-case.
- **Deterministic SELECT (Median-of-Medians):**
  - Pivot M^* is the median of $n/5$ group medians computed recursively.
  - Guarantees a 30-70 split: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$.
  - **Complexity:** $\Theta(n)$ guaranteed worst-case.

- **Randomized QuickSort:** Pick pivot p at random; recurse on *both* sides.
- **Recurrence:** $T(n) = T(r-1) + T(n-r) + \Theta(n)$.
- **Complexity:**
  - **Worst-case:** $\Theta(n^2)$ (pivot is always min or max).
  - **Expected/Best:** $\Theta(n \log n)$.

Lower Bound for Comparison Sorting

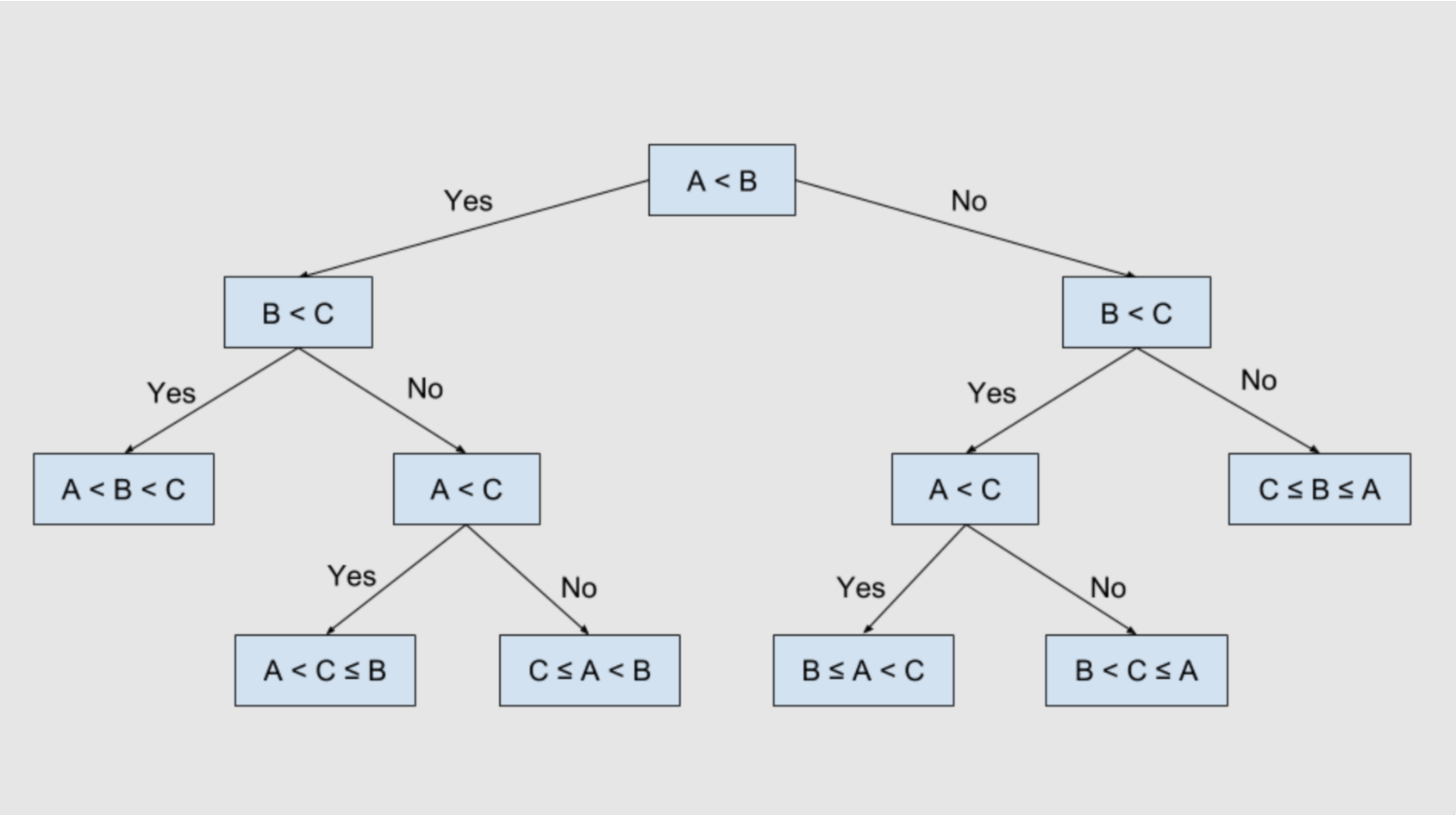
Question: Can we sort faster than $\Theta(n \log n)$ if we only use comparisons?

The Decision Tree Model:

1. Any sorting algorithm that uses only comparisons can be viewed as a **binary decision tree**.
2. Each **internal node** represents a comparison between two elements (e.g., $a_i < a_j$).
3. Each **leaf** represents a possible final sorted order (permutation).
4. The **height** of the tree represents the number of comparisons in the worst case.

.

Decision Tree for Sorting



Example of a decision tree for sorting 3 elements.

Each internal node represents a comparison, and leaves represent sorted orders.

Lower Bound for Comparison Sorting

The Proof Logic:

- There are $n!$ possible ways to order n elements. To be correct, the tree must have at least $n!$ leaves.
- A binary tree of height h can have at most 2^h leaves.
- Therefore: $2^h \geq n!$
- Taking the logarithm: $h \geq \log(n!) = \Theta(n \log n)$.

Result: Every comparison-based sorting algorithm must perform $\Omega(n \log n)$ comparisons in the worst case.

Counting Sort: Non-Comparison Sorting

Problem: Sort an array $A[1 \dots n]$ where every value is an integer in the range $[1 \dots M]$.

1. Create a frequency array C of size M, initially all zeros.
2. **Count:** Traverse array A. For each value x encountered, increment $C[x]$ by 1.
3. **Reconstruct:** Iterate through the frequency array C from 1 to M:
   - For each index j, write the value j back into array A exactly $C[j]$ times.

Note: This algorithm is efficient because it does not compare elements against each other. Instead, it uses the *values* as indices.

Counting Sort: Example

Input: $A = [4, 2, 2, 8, 3, 3, 1]$, where $M = 8$.

Step 1: Count Occurrences

- We see one '1', two '2's, two '3's, one '4', and one '8'.
- Frequency Array C:

Index	1	2	3	4	5	6	7	8
Count	1	2	2	1	0	0	0	1

Step 2: Reconstruct Sorted Array

- Write '1' (one time) $\rightarrow [1, \ldots]$
- Write '2' (two times) $\rightarrow [1, 2, 2, \ldots]$
- Write '3' (two times) $\rightarrow [1, 2, 2, 3, 3, \ldots]$
- ... and so on.

Final Output: $A = [1, 2, 2, 3, 3, 4, 8]$

Counting Sort: Analysis

Time Complexity:

- **Initializing C:** Takes $\Theta(M)$ time.
- **Counting Elements:** We scan n elements, taking $\Theta(n)$ time.
- **Writing Output:** We iterate through M counts and write n total values, taking $\Theta(n + M)$ time.

Total Runtime: $\Theta(n + M)$.

Takeaways:

- Counting sort is **linear** if the range M is not much larger than n (i.e., $M = O(n)$). It is much faster than the $\Omega(n \log n)$ lower bound for comparison sorts.
- **Drawback:** If M is very large (e.g., $M = n^2$), the space and time costs become inefficient.

Review Data Structures

- Hashing, Hash Maps
- Linked Lists
- Stacks and Queues
- Binary Search Trees (BSTs)
- balanced BSTs: Red-Black , 2- 3 trees
- Heaps and Priority Queues
- Graphs

Simple Queue

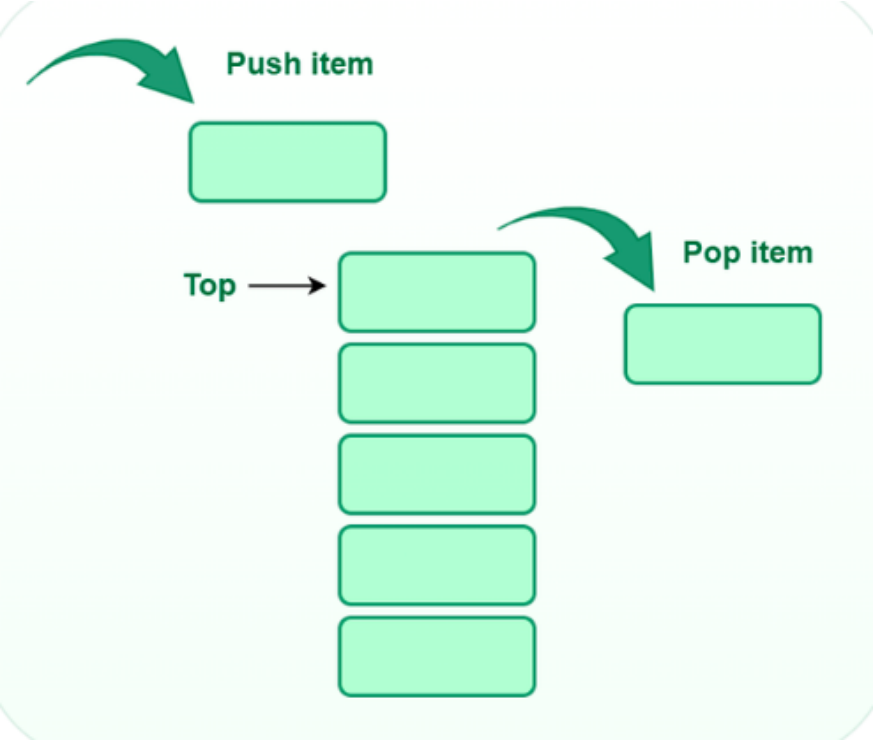
Retrieves elements in the order they were added- First-In, First-Out ("FIFO")

- **add(item):** aka "enqueue" - Add an element to the back.
- **remove():** aka "dequeue" - Remove the front element and return.
- **peek():** Examine the front element without removing it.
- **size():** Determine how many items are stored in the queue.
- **isEmpty():** Returns true if there are no items in the queue, false otherwise.

All the operations take $O(1)$ time.

Stack

A collection based on the principle of adding elements and retrieving them in the opposite order. Last-In, First-Out ("LIFO")



- **push(item)**: Add an element to the top of the stack.
- **pop()**: Remove the top element and return it.
- **peek()**: Examine the top element without removing it.
- **size()**: Determine how many items are in the stack.
- **isEmpty()**: Return `true` if there are no items in the stack, and `false` otherwise.

Priority Queue

- A Priority Queue sends the highest-priority message first.
- Supported operations:
  - `Q.add(x)`: Add element `x` to the queue.
  - `Q.find-max()`: Return the highest priority element.
  - `Q.delete-max()`: Remove the highest priority element.
- Use a Heap for efficient priority queue operations.

Max-Heap Operations

- A Max-Heap supports the following operations:
  - `Heap.add(x)`: Add `x` to the heap in $O(\log n)$.
  - `Heap.find-max()`: Return the maximum element in $O(1)$.
  - `Heap.delete-max()`: Remove the maximum element in $O(\log n)$.
- Heaps are used for:
  - Efficient priority queues
  - Sorting algorithms like Heap Sort

Heap Sort Algorithm

- Heap Sort is an efficient sorting algorithm with complexity $O(n \log n)$.
- Steps:
  - Build a Max-Heap from the array: $O(n)$.
  - Repeatedly extract the maximum element from the heap and place it at the end: $O(\log n)$ per element.
- Total complexity: $O(n \log n)$.

Heap Sort: Example

Example: Sorting $A = [4, 10, 3, 5, 1]$

- Step 1: Build a max-heap $\rightarrow$ Heap becomes [10, 5, 3, 4, 1].
- Step 2: Extract max and rebuild heap:
  - Remove 10 $\rightarrow$ Heap [5, 4, 3, 1] (Sorted: [10]).
  - Remove 5 $\rightarrow$ Heap [4, 1, 3] (Sorted: [10, 5]).
  - Remove 4 $\rightarrow$ Heap [3, 1] (Sorted: [10, 5, 4]).
  - Remove 3 $\rightarrow$ Heap [1] (Sorted: [10, 5, 4, 3]).
  - Remove 1 $\rightarrow$ Heap empty (Sorted: [10, 5, 4, 3, 1]).
- Final output: [1, 3, 4, 5, 10].

Sorting a k-Sorted Array

- An array is **k-sorted** if every element is at most k positions away from its correct position.
- Algorithm:
  - Build a Min-Heap with the first $k + 1$ elements.
  - For each element in the array, extract the minimum and insert the next element.
- Total complexity: $O(n \log k)$, which is better than $O(n \log n)$ when k is small.

Sorting a k-Sorted Array: Example

Example: Sorting $A = [6, 5, 3, 2, 8, 10, 9]$ with $k = 3$

- Step 1: Build a min-heap with the first $k + 1 = 4$ elements $\rightarrow$ Heap becomes [2, 5, 3, 6].
- Step 2: Extract min and insert next element:
  - Remove 2, insert 8 $\rightarrow$ Heap [3, 5, 6, 8].
  - Remove 3, insert 10 $\rightarrow$ Heap [5, 8, 6, 10].
  - Remove 5, insert 9 $\rightarrow$ Heap [6, 8, 10, 9].
  - Remove 6 $\rightarrow$ Heap [8, 9, 10].
  - Remove 8 $\rightarrow$ Heap [9, 10].
  - Remove 9 $\rightarrow$ Heap [10].
  - Remove 10 $\rightarrow$ Heap empty.
- Final output: [2, 3, 5, 6, 8, 9, 10].

The Two Sum Problem

Given an array A of length n and a target sum T, find indices i, j such that $A[i] + A[j] = T$, or return NIL if no such pair exists.

Brute Force Approach:

- Compare all pairs (i, j) where $i < j$.
- Time complexity: $O(n^2)$.

Optimized Approach: Can we do better than $O(n^2)$?

Idea: If we could search for $T - A[i]$ faster, we can reduce the complexity.

Faster Search: Sorting and Binary Search

- Sort the array (Merge Sort: $O(n \log n)$).
- For each $A[i]$, use Binary Search to find $T - A[i]$ in $O(\log n)$.
- Total complexity: $O(n \log n)$.

Can we search even faster than $O(\log n)$?

Faster Search: Counting Sort for Lookup

- If all elements are natural numbers, we can use Counting Sort to preprocess in $O(n + M)$, where M is the maximum element.
- Store occurrences in an array C where $C[x]$ is the count of x in A.
- Search for any number x in $O(1)$ time.
- Total time: $O(n + M)$.

Limitation: Large values of M make this infeasible.

Introducing Hash Maps (Dictionaries)

Motivation: We need a data structure that allows:

- Fast lookup ($O(1)$ on average).
- Efficient insert and delete operations.
- Flexibility to store arbitrary numbers, not just bounded integers.

Solution: Hash Maps (Dictionaries)

- Store (key, value) pairs.
- Search, insert, and delete in $O(1)$ average time.

Problem: Two Sum

Input: An array `A` of length `n` and a target sum `T`.

Output: Return indices `i` and `j` such that `A[i] + A[j] = T`, or NIL if no such pair exists.

Algorithm

1. Initialize an empty dictionary `D`.
2. **For** each index `i` in `A`:
   - Let `complement = T - A[i]`.
   - **If** `complement` exists in `D`, return the indices `D[complement]` and `i`.
   - **Else** add `A[i]` to `D`.
3. **Return** NIL if no valid pair is found.

Time Complexity: $O(n)$ (on average, assuming a good hash function).

Problem: Remove Duplicates

Input: An array `A` of length `n`.

Output: An array `B` containing unique elements from `A`, preserving order.

Algorithm

1. Initialize an empty array `B`.
2. Initialize an empty dictionary `D`.
3. **For** each element `A[i]`:
    - **If** `A[i]` is not in `D`:
        - Add `A[i]` to `D`.
        - Append `A[i]` to `B`.

Time Complexity: $O(n)$ (on average, using a hash set).

Problem: Equal Sums

Input: An array `A` with unique elements.

Output: Indices `i, j, p, q` such that `A[i] + A[j] = A[p] + A[q]`.

Algorithm

1. Initialize an empty dictionary `D`.
2. **For** each index `i`:
   - **For** each index `j` where `j > i`:
     - Compute `sum = A[i] + A[j]`.
     - **If** `sum` exists in `D`, return `i, j, D[sum].first, D[sum].second`.
     - **Else** store `(i, j)` in `D` under `sum`.

Time Complexity: $O(n^2)$.

Problem: Equal Arrays

Input: Two arrays `A` and `B` of length `n`.
Output: Return TRUE if `A` and `B` contain the same elements with identical frequencies.

Algorithm

1. **If** `len(A)`$\neq len(B), return FALSE$.
2. Initialize dictionaries `D_A` and `D_B`.
3. **For** each element `A[i]`:
   - Increment count of `A[i]` in `D_A`.
4. **For** each element `B[i]`:
   - Increment count of `B[i]` in `D_B`.
5. **For** each key in `D_A`:
   - **If** counts do not match in `D_B`, return FALSE.
6. **Return** TRUE.

Time Complexity: $O(n)$.

Problem: Top K Largest Elements

Input: An array `A` of length `n` and an integer `k`.

Output: Return the k largest elements in `A`.

Algorithm

1. Initialize a min-heap `H` of size `k`.
2. **For each** element `x` in `A`:
   - **If** `H` has fewer than `k` elements, insert `x` into `H`.
   - **Else if** `x` is greater than the smallest element in `H`:
     - Remove the smallest element from `H` and insert `x`.
3. **Return** the elements in `H`.

Time Complexity: $O(n \log k)$

Problem: Kth Largest Element in a Stream

Input: A stream of integers and an integer `k`.

Output: Return the kth largest element in the stream at any point.

Algorithm

1. Initialize a min-heap `H` to store up to `k` elements.
2. **For each** element `x` in the stream:
   - **If** `H` contains fewer than `k` elements, insert `x` into `H`.
   - **Else if** `x` is greater than the smallest element in `H`:
     - Remove the smallest element from `H` and insert `x`.
3. **Return** the root of `H`, which contains the kth largest element.

Time Complexity: $O(n \log k)$

Problem: Reorganize String

Input: A string `S` of length `n`.
Output: Return a reorganized string such that no two adjacent characters are the same. If not possible, return an empty string.

Algorithm

1. Initialize a frequency map `D` for all characters in `S`.
2. Add all characters to a max-heap `H` based on their frequency.
3. Initialize an empty result string `R`.
4. **While** `H` is not empty:
   - Remove the character `x` with the highest frequency from `H`.
   - **If** `R` is empty or `x` is not the last character:
     - Append `x` to `R`.
     - Decrease the frequency of `x`, and if still positive, reinsert into `H`.
   - **Else:** Repeat the process for the next most frequent character.
5. **Return** `R`, or an empty string if the condition isn't met.

Problem: Top K Frequent Elements

Input: An array `A` of length `n` and an integer `k`.
Output: Return the k most frequent elements.

Algorithm

1. Use a dictionary `D` to count the frequency of each element in `A`.
2. Insert each element into a min-heap of size `k`, keeping the most frequent elements at the top.
3. If the heap exceeds size `k`, remove the least frequent element.
4. **Return** the elements in the heap after processing all elements.

Problem: Top K Frequent Words

Input: An array of words `W` of length `n` and an integer `k`.
Output: Return the k most frequent words in `W`.

Algorithm

1. Count the frequency of each word using a dictionary `D`.
2. Insert each word into a min-heap `H`. If `H` has more than `k` elements, remove the word with the lowest frequency.
3. Continue this process until all words are processed.
4. **Return** the words in `H`, which contains the k most frequent words.

Problem: Two Sum

Input: An array `A` of length `n` and a target sum `T`.

Output: Return indices `i` and `j` such that `A[i] + A[j] = T`, or NIL if no such pair exists.

Algorithm

1. Initialize an empty dictionary `D`.
2. **For** each index `i` in `A`:
   - Let `complement = T - A[i]`.
   - **If** `complement` exists in `D`, return the indices `D[complement]` and `i`.
   - **Else** add `A[i]` to `D`.
3. **Return** NIL if no valid pair is found.

Problem: Remove Duplicates

Input: An array `A` of length `n`.

Output: Return an array with unique elements from `A`.

Algorithm

1. Initialize an empty array `B` and an empty dictionary `D`.
2. **For each** element `A[i]`:
   - **If** `A[i]` is not in `D`, add `A[i]` to both `B` and `D`.
3. **Return** `B` as the result.

Problem: Equal Arrays

Input: Two arrays `A` and `B` of length `n`.
Output: Return TRUE if `A` contains the same elements as `B` (with identical frequencies).

Algorithm

1. Initialize dictionaries `D_A` and `D_B` to count frequencies.
2. Count the frequency of elements in both arrays.
3. **Return** TRUE if both dictionaries are identical, otherwise return FALSE.

Subarray Sum Equals K

Input: An array `A` of length `n` and a target integer `k`.

Output: The number of subarrays that sum to `k`.

Algorithm

1. Initialize a dictionary `D` to track cumulative sums. Set `D[0] = 1`.
2. Set `cumulative_sum = 0` and `count = 0`.
3. **For each** element `A[i]`:
   - Add `A[i]` to `cumulative_sum`.
   - If `cumulative_sum - k` exists in `D`, increment `count` by `D[cumulative_sum - k]`.
   - Update `D[cumulative_sum]` with its new frequency.
4. Return `count`.

Longest Consecutive Sequence

Input: An unsorted array `A`.

Output: Length of the longest sequence of consecutive elements.

Algorithm

1. Convert `A` into a set `S` for quick lookups.
2. Initialize `longest_streak = 0`.
3. **For each** element `x` in `S`:
   - If `x-1` is not in `S`, set `current_streak = 1`.
   - While `x + 1` exists in `S`, increment `current_streak`.
   - Update `longest_streak`.
4. Return `longest_streak`.

Group Anagrams

Input: A list of strings `S`.

Output: Group all anagrams together.

Algorithm

1. Initialize an empty dictionary `D`.
2. **For each** string `s` in `S`:
   - Sort `s` to get `sorted_s`.
   - Append `s` to `D[sorted_s]` if it exists, otherwise initialize a new list for it.
3. Return the grouped anagrams from the values of dictionary `D`.

Find All Anagrams in a String

Input: Two strings, `s` and `p`.

Output: Return the start indices of all anagrams of `p` in `s`.

1. Create dictionaries `D_s` and `D_p` to count character frequencies in `s` and `p`.
2. Populate `D_p` with the frequency of characters in `p`.
3. Slide a window of size `len(p)` across `s`:
   - Add each character to `D_s`.
   - If the window exceeds `len(p)`, remove the oldest character.
   - If `D_s` matches `D_p`, record the start index.
4. Return the list of start indices.

Introduction to Hashing

Key Idea

- Use a **hash table** $T[1:m]$.
- Use a **hash function** $h : N \to [1:m]$ to map elements to indices.

Efficiency

Search time: $O(1)$ under ideal conditions.

Collision Problem in Hashing

What is a Collision?

- Two elements map to the same index.
- Example: $h(72) = h(16)$.

Collision Example

Scenario

- $A[4] = 72$, $A[3] = 16$.
- If both hash to the same index:

$$T[h(72)] = T[h(16)]$$

Impact

Collisions must be handled to maintain efficiency.

Why Are Collisions Inevitable?

Fundamental Issue

- Hash functions map infinite numbers to a finite set.
- By the **pigeonhole principle**, some indices must hold multiple values.

How to Limit Collisions?

Key Observations

- **Fixed hash functions** always have worst-case inputs.
- There exists an input where **every** element collides.

Using Random Hash Functions

Power of Randomness

- Instead of a fixed h, choose h randomly from a family of hash functions.
- This reduces the likelihood of worst-case behavior.

Random Hash Functions

Key Idea

Instead of using a fixed hash function, we select one randomly from a family of hash functions H.

Example Hash Family

$$H = \{h_a(x) = (a \cdot x \mod m) + 1 \mid 1 \leq a \leq m - 1\}$$

- Example functions:

$$h_1(x) = (x \mod m) + 1, \quad h_2(x) = (2x \mod m) + 1$$

Random Hash Functions

Why Randomization?

Choosing $h \in H$ **after** the input array is fixed prevents adversarial inputs from causing worst-case behavior.

Uniform Hash Family

Definition

A hash family H is **uniform** if for any input x and index i,

$$\Pr_{h \in H}(h(x) = i) = \frac{1}{m}$$

Intuition

- Each input x is mapped to any index in T with equal probability.
- Ensures that elements are spread evenly across the table.

Universal Hash Family

Definition

A hash family is **universal** if for any two distinct values $x \neq y$,

$$\Pr_{h \in H}(h(x) = h(y)) \leq \frac{1}{m}$$

Why Is This Important?

- The probability of two elements colliding is at most $\frac{1}{m}$.
- Universal hashing significantly reduces expected collisions.

Expected Number of Collisions

Collision Analysis

Using a universal hash family H with a table of size $m = n$, the expected number of collisions is:

$$\mathrm{E}[\text{number of collisions}] = \sum_{i \neq j} \Pr(h(A[i]) = h(A[j])) = n$$

Key Takeaway

On average, each element will experience at most one collision.

Near-Universal Hash Family

Definition

A hash family is **near-universal** if for any $x \neq y$,

$$\Pr_{h \in H}(h(x) = h(y)) \leq \frac{2}{m}$$

Why Use Near-Universal Hashing?

- Allows a higher probability of collisions but keeps it manageable.
- Easier to construct compared to fully universal hash families.

Example of Near-Universal Hash Functions

Hash Family

$$H = \{h_a(x) = ((a \cdot x \mod p) \mod m) + 1 \mid 1 \leq a \leq m - 1\}$$

where p is a prime larger than any element in A.

Handling Collisions: Chaining

Chaining Method

- Each index $T[i]$ in the hash table stores a linked list.
- If multiple elements hash to the same index, they are stored in the list.

Algorithm

1. Compute $h(A[i])$ for each element.
2. Insert $A[i]$ into the linked list at $T[h(A[i])]$.

Searching

To find x**,** compute $h(x)$ and perform a linear search in $T[h(x)]$.

Runtime Analysis of Chaining

Time Complexity

- **Construction:** $O(n + m)$ (hash computation + insertions).
- **Search:** $O(1 + \ell(x))$, where $\ell(x)$ is the linked list length.

Expected Worst-Case Runtime

Expected Length of Linked List

$$E[\ell(x)] = \sum_{i=1}^{n} \Pr(h(A[i]) = h(x)) = \frac{2n}{m}$$

What This Means?

- If $m = O(n)$, the expected search time remains $O(1)$.
- Hashing with chaining is efficient in practice.

Problem: Find-1

Problem Statement

Input: An array A of 0s and 1s, where at least 25% of the values are 1.

Output: Return an index i such that $A[i] = 1$.

Deterministic Algorithm

1. Loop through the array from start to end.
2. If $A[i] = 1$, **return** i.

Time Complexity

- **Average:** $O(1)$ (if the first 1 appears early).
- **Worst-case:** $\Omega(n)$ (if the first 1 is near the end).

Randomized Algorithm for Find-1

Algorithm

Repeat until a 1 is found:

1. Select a random index $i \in [1, n]$.
2. **If** $A[i] = 1$, **return** i.

Expected Runtime

- **Expected:** $O(1)$ (since each iteration has a 25% success rate).
- On average, it takes 4 iterations to find a 1.

Worst-Case Runtime

- Theoretically infinite (if unlucky and only selects 0s).

Comparison of Algorithms

Types of Algorithms

- **Perfect Deterministic:** Always fast (e.g., merge sort).
- **Deterministic Heuristic:** Fast for most inputs but slow for some (e.g., deterministic Find-1).
- **Randomized:**
  - **Las Vegas:** Always correct and usually fast (e.g., randomized Find-1).
  - **Monte Carlo:** Always fast, but only correct with high probability.

Key Insights

- Deterministic heuristics can fail for specific bad inputs.
- Randomized algorithms are harder to sabotage, even with adversarial input.

Randomization in Hash Maps

Deterministic Hash Maps

- Provide good performance for most inputs.
- However, adversarial inputs can cause consistently poor performance.
- If input distribution is unknown, a deterministic heuristic may fail.

Randomized Hash Maps

- Perform well on all inputs with high probability.
- Offer stronger guarantees, working efficiently for 99.999% of cases.